University of Sheffield

COM1001 SPRING SEMESTER

Professor Phil McMinn

p.mcminn@sheffield.ac.uk

Automated Testing

An Introduction to Testing

Testing has always been part of software development

… when you wrote your first program, you almost certainly tried it out with some sample data

For a long time, this was the state of the art in industrial practice!

In the early 2000s, software development practices started to change

Software systems got too big and too complex for manual testing to remain an effective and efficient way to ensure they were working and **remained working**

Testing at the Speed of Modern Software Development

Software systems are growing larger and evermore complex.

A typical application or service at Google, for example, is made up of thousands or millions of lines of code.

The ability for humans to **manually validate every behaviour in a system** has been **unable to keep pace with the explosion of features and platforms in most software**.

Testing at the Speed of Modern Software Development

Imagine what it would take to manually test the functionality of Google search – every time the code was changed.

… not just web search, but images, flights, movie times etc.

Then multiply that for every language, country, and device that must be supported.

Then add in factors like accessibility and security.

Manual testing does not scale. We need automation.

Developer-Driven Automated Testing

The idea of coding automated tests (e.g., in JUnit) as a means of improving productivity and velocity may seem antithetical.

After all, the act of writing tests can take just as long (if not longer!) than implementing a feature in the first place … right?

On the contrary!

> In industry, investing in software tests provides several key benefits to developer productivity.

Less Debugging

Tested code has fewer defects when it is submitted.

Crucially, it also has fewer defects throughout its existence – since code tends to be updated during its lifetime.

> … it will be changed by other teams and even automated code maintenance systems.
>
> Changes to code, or its dependencies, can be quickly detected by an automated test and rolled back before the problem reaches production.

Increased Confidence in Changes

Projects with good tests can be modified with confidence since all the important behaviours of their projects are continuously being verified.

These projects *encourage* **refactoring** (more on this next week!).

After a change, we can re-run the automated tests to ensure we didn’t break any of the existing functionality.

Improved Documentation

Software documentation is notoriously unreliable!

Clear, focused tests that exercise one behaviour at a time function as executable documentation.

Thoughtful Design

Writing tests for new code is a practical means of exercising the API design of the code itself.

If new code is difficult to test, it is often because the code being tested has too many responsibilities or difficult-to-manage dependencies.

Well-designed code should be modular, avoiding tight coupling and focusing on specific responsibilities.

Fixing design issues early means less rework later.

Fast, High Quality Releases

With a healthy automated test suite, teams can release new versions of their application with confidence.

Many large projects, involving hundreds of engineers and thousands of code changes submitted every day, involve very short release cycles – often every day.

This would not be possible without automated testing.

Benefits of an Automated Test Suite

1. Less Debugging
2. Increased Confidence in Changes
3. Improved Documentation
4. Thoughtful Design
5. Allows for Fast, High Quality Software Releases

What is *Testing*?

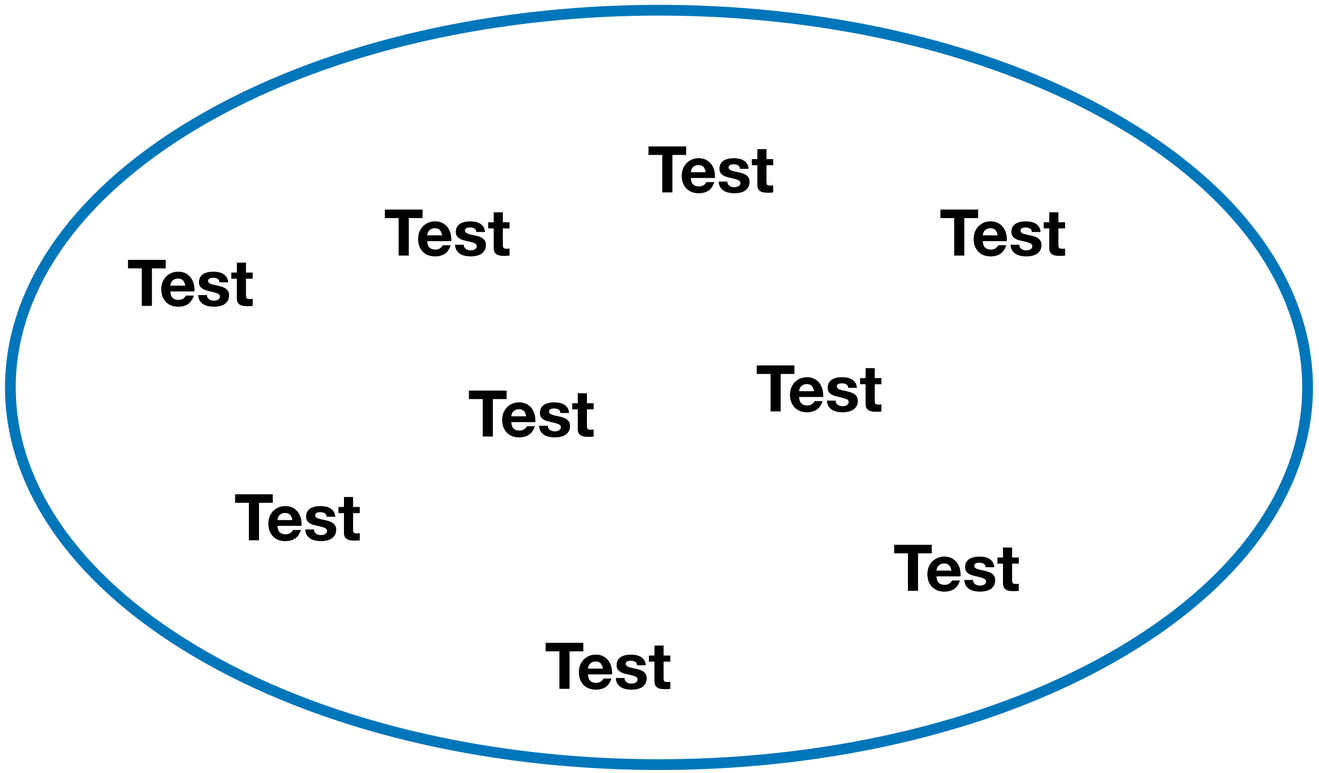
What is a *Test Case*?

What is a *Test Suite*?

Ingredients of a Test Case

1. The inputs needed to put the software into the right state for the test
2. The actual test case inputs
3. The expected results of the test
4. Reset of the system state

A Test Suite – A Set of Tests



Ideally, the tests can be executed in any order